

MERGEOver: Post-Training Token Merging for Recursive Vision Transformers

Junseo Kim

Computer Architecture for Embedded
Systems, University of Twente
Enschede, The Netherlands
j.kim-5@student.utwente.nl

Uraz Odyurt

Faculty of Engineering Technology,
University of Twente
Enschede, The Netherlands
u.odyurt@utwente.nl

Amirreza Yousefzadeh

Computer Architecture for Embedded
Systems, University of Twente
Enschede, The Netherlands
a.yousefzadeh@utwente.nl

Abstract

Vision Transformers (ViTs) demonstrate exceptional performance in computer vision but suffer from large parameter counts and quadratic computational complexity, severely limiting their deployment on resource-constrained edge hardware. While recursive weight-sharing reduces parameter counts and token merging mitigates computational and memory bottlenecks, integrating these two paradigms without costly retraining is non-trivial, leaving this intersection largely unexplored. We propose MERGEOver, a post-training approach that integrates Token Merging (ToMe) into the recursively weight-shared Sliced Recursive Transformer (SReT). Through an *Unmerge* tracking stack, constraint-safe merge-rate adjustment, and synchronised token-mass tracking across spatial permutations, MERGEOver resolves the spatial and merging constraints of this integration. We further employ a stage-wise single-shot schedule that performs token reduction at the first block of each stage and maintains a fixed sequence length throughout its subsequent recursive iterations. Benchmarked on ImageNet-1K, our selected configuration reduces top-1 accuracy by 1.47 percentage points. On the GPU, it reduces peak activation memory by 37.3% and 38.4% at batch sizes 1 and 16, while throughput decreases by 21.7% at batch size 1 but increases by 21.7% at batch size 16. On a Raspberry Pi 5 (ARM CPU), it reduces latency by 2.4% and 17.6% at batch sizes 1 and 16. These results show that MERGEOver can recover a meaningful part of the throughput and memory cost that recursive weight-sharing introduces, without retraining, and provides a baseline for combining token merging with hierarchical recursive transformers.

Keywords

Vision transformers, Model compression, Recursive weight-sharing, Token merging, Edge AI

1 Introduction

The self-attention mechanism of the Transformer architecture [1] has revolutionised computer vision through the introduction of the Vision Transformer (ViT) [2]. However, this architecture results in massive parameter counts and quadratic computational complexity, $O(N^2)$, with respect to sequence length, N , limiting practical deployment on resource-constrained edge hardware [3, 4]. To reduce parameter counts, recursive weight-sharing paradigms have been explored, which repeatedly route the input sequence through a single transformer block, applying identical weights across multiple passes [5, 6, 7]. However, this approach results in increased computational complexity and Peak Activation Memory (PAM).

To explicitly target these computational and memory bottlenecks, token reduction paradigms have emerged, aiming to reduce the token sequence during inference. Modern strategies include token pruning [8, 9], token merging [10, 11], and token fusion [12]. Amongst these, Token Merging (ToMe) [10] stands out as a promising post-training framework.

Recursive weight-sharing and token reduction have developed as independent lines of work, and their combination has not been studied. Moreover, while alternative efficient architectures exist, they rely on re-training phases, distillation, or structural redesigns [3, 4, 13, 14, 15]. This creates a literature gap for post-training compression methods. We address this gap by proposing a post-training compression approach that combines recursive weight-sharing with token merging. We validate this integration through a concrete case study that integrates the Sliced Recursive Transformer (SReT) [7] with ToMe [10]. Our developed source code is openly available [16].

Naive integration introduces severe architectural friction. First, token reduction through merging violates the rigid 2D spatial grid required by SReT’s hierarchical convolutional pooling layers. Second, unconstrained merging can violate the strict group-size boundaries inherent to the Sliced Group Self-Attention (SGA) of SReT. Third, SReT’s spatial permutations break the tracking necessary for the proportional attention of ToMe. Resolving these incompatibilities is mandatory for a successful integration.

Contributions. Specifically, we make the following contributions:

- The formulation of MERGEOver, a post-training approach that enables token merging within a hierarchical recursive transformer, without retraining.
- An analysis of token reduction scheduling strategies, culminating at a stage-wise single-shot reduction strategy compatible with SReT.
- A cross-platform evaluation of MERGEOver against an uncompressed recursive baseline on the ImageNet-1K dataset [17].

Following this introduction, Section 2 contextualises our work within literature, while Section 3 elaborates the SReT and ToMe integration. Section 4 details the experimental setup, complemented by Section 5 covering benchmarking results and discussion. Sections 6 and 7 point out the limitations and provide our concluding remarks.

2 Background and related work

We review the existing literature and foundational concepts related to MERGEOver.

2.1 Vision Transformers and spatial pyramids

Modern ViTs stem from the Transformer architecture [1], computing global dependencies via scaled dot-product attention over Queries (Q), Keys (K), and Values (V):

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (1)$$

where d_k is the scaling dimension. ViT [2] adapted this to computer vision by flattening 2D images into 1D patch sequences. However, standard ViTs maintain a strictly isotropic layout where sequence length and channel capacity remain uniform, resulting in a quadratic computational complexity relative to sequence length.

To mitigate this bottleneck, hierarchical spatial pyramids introduce progressive downsampling. While Swin [18] and PVT [19] employ localised attention or spatial reduction, PiT [20] directly integrates CNN-based convolutional pooling. This effectively models complex spatial hierarchies, establishing the exact architectural blueprint inherited by recursive variants such as SReT [7].

2.2 Recursive weight-sharing

Weight-sharing minimises parameters by reusing weights via recurrence. Pioneered by the Universal Transformer [5], this paradigm limits parameter counts while allowing arbitrary computational depth. Early vision adaptations like MiniViT [6] applied layer-wise weight-multiplexing to simulate depth, but lacked true parametric recursion.

Pure spatial recurrence within a hierarchical architecture was realised by SReT [7]. Built on PiT’s [20] convolutional pyramid blueprint, SReT routes sequences recursively through identical blocks with frozen parameters across consecutive loops.

While SReT is highly parameter-efficient, repeating self-attention creates a severe computational bottleneck, and routing tokens through a shared parameter space accelerates feature densification. This severely penalises throughput and increases PAM, necessitating sequence-level compression.

2.3 Token reduction

To alleviate the quadratic complexity of ViTs, token reduction paradigms minimise spatial redundancy during inference via token pruning [8, 9], token merging [10, 11], or token fusion [12].

Token pruning permanently discards patches. However, methods like DynamicViT [8] require retraining, while post-training alternatives like EViT [9] rely heavily on the *extra class token* (CLS), which is incompatible with SReT’s [7] global average pooling. Crucially, permanent deletion prevents reconstructing the rigid 2D grid required for inter-stage convolutional pooling.

In contrast, ToMe [10] is a post-training module that merges similar tokens via Bipartite Soft Matching (BSM). To preserve proportional attention after merging, ToMe tracks the number of original patches represented by each token using a token-mass vector (s). The logarithm of the key-token masses is added to the attention logits before the softmax:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \log(s_k)\right)V. \quad (2)$$

While variants like PiToMe [11] optimise matching, ToMe’s reversibility is uniquely valuable for hierarchical networks. Along

with a merge operation, BSM enables an *Unmerge* operation that duplicates features to restore original sequence length. This infrastructure provides the exact mechanism to satisfy the spatial layout constraints of SReT’s inter-stage convolutional pooling.

Token fusion frameworks like ToFu [12] hybridise these paradigms. However, combining destructive pruning with irreversible merging destroys bi-directional tracking. Lacking a structural Unmerge function, fusion remains incompatible with SReT’s hierarchical pyramid.

2.4 Alternative efficiency paradigms

Beyond recursive weight-sharing and token reduction, several alternative paradigms target structural efficiency. Architectural redesigns (MobileViT [3], EfficientViT [4]), reparameterisations (FastViT [13], RepViT [14]), and structured pruning (X-Pruner [21]) increase throughput by modifying layouts or cutting channels. However, their reliance on intensive retraining prevents post-hoc application to pre-trained models.

Conversely, implementation-level and precision-reduction frameworks optimise hardware execution without altering architecture. Hardware-aware kernels like FlashAttention [22] bypass memory bottlenecks via GPU SRAM tiling, but leave spatial token redundancy untouched. Similarly, post-training quantisation (FQ-ViT [23]) compresses weights and activations into low-bit formats during inference. By optimising memory access and numerical precision rather than parameter counts or sequence lengths, these approaches represent entirely complementary mechanisms that can integrate alongside our proposed approach.

2.5 Multi-axis compression

ViT optimisation has shifted from isolated, single-axis methods toward multi-axis joint compression. For instance, MADTP++ [15] unifies structural weight pruning and dynamic token pruning but requires retraining, preventing inference-only deployment. To bypass this overhead, post-training strategies like QUOTA [24] merge low-bit quantisation with deterministic sequence pruning.

While highly efficient for isotropic ViTs, these paradigms rely exclusively on destructive, non-reversible operations. Crucially, the intersection of recursive weight-sharing and dynamic token merging remains unexplored. MERGEOver fills this void using a post-training integration of these two axes.

3 Methodology and implementation

Figure 1 illustrates the complete technical integration of SReT and ToMe, detailing how tokens are reduced throughout the complete forward pass. The inner workings of different steps are described.

3.1 Spatial layout constraints

SReT [7] inherits its structure directly from PiT [20], which utilises a spatial pyramid design. Unlike standard ViTs that split an image into non-overlapping patches using a linear projection, SReT applies overlapping convolutional embeddings to generate its initial token sequence. As depicted in Figure 2, these tokens are then processed through three distinct stages. Each stage contains a set of Transformer blocks that are executed recursively. Between these stages, convolutional pooling layers decrease the spatial sequence

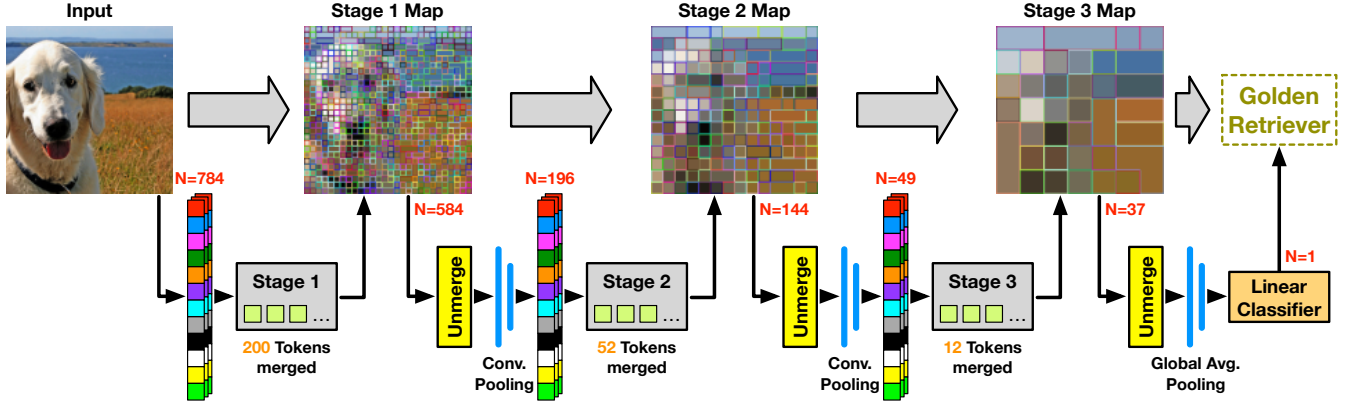


Figure 1: An overview of the proposed MERGEOver approach, integrating the recursive weight-sharing of SReT with the token merging of ToMe. The initial token sequence of length $N = 784$ is reduced using a stage-wise single-shot schedule. Crucially, the spatial layout is reconstructed prior to each inter-stage convolutional pooling layer through the *Unmerge* block.

length while increasing channel depth. After the final stage, global average pooling compresses the tokens for final classification.

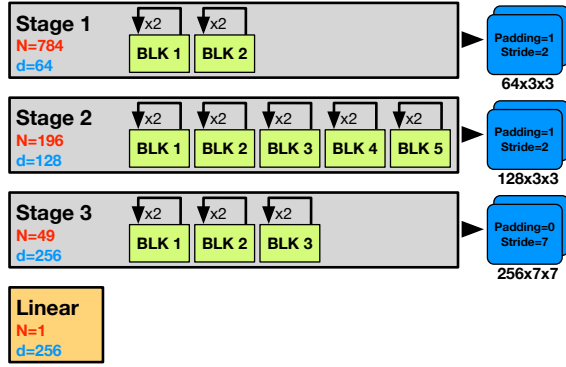


Figure 2: The hierarchical spatial pyramid architecture of SReT, covering the 3 stages and the Linear classifier stage from Figure 1. Each stage involves Transformer blocks (in orange) with $\times 2$ recursive iterations, leading to convolutional pooling layers (in blue). Token sequence length (N) and channel depth (d) values are provided throughout. Dynamic token reduction must satisfy the strict spatial downsampling of convolutional pooling layers.

This hierarchical design means that after tokens exit the Transformer blocks of a given stage, they must pass through an inter-stage convolutional pooling layer. Since convolutional kernels operate on structured 2D spatial grids, e.g., 14×14 , they expect a sequence length that can be perfectly reshaped into a square tensor. However, standard token reduction eliminates an arbitrary number of tokens, destroying this perfect square property. Consequently, the sequence becomes structurally invalid for the convolutional kernels, triggering a dimensional clash at the end of the stage.

The BSM algorithm of ToMe [10] provides a unique mechanism to resolve this clash. During a merge operation, ToMe returns both a

merging function and a corresponding *Unmerge* function. Crucially, this *Unmerge* function does not reverse the aggregation of the features. Instead, it restores the spatial dimensions of the sequence by duplicating the merged features back into their original spatial coordinates. This implies that regardless of how aggressively a sequence is compressed, the initial sequence length can be restored, provided that the *Unmerge* functions are stored. This property is highly beneficial for SReT, offering a direct solution to satisfy the geometric requirements of inter-stage convolutional pooling.

To utilise this capability, we implement an *Unmerge stack* as shown in Figure 3. As the token reduction schedule dynamically executes merge operations within a stage, the corresponding *Unmerge* operations are pushed onto a last-in-first-out stack. Before the compressed sequence reaches the inter-stage convolutional pooling layer, the tensor is built back up to its original dimensions by applying the stored *Unmerge* operations in strict reverse order.

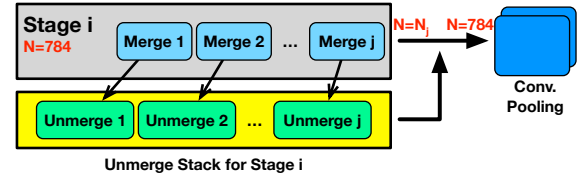


Figure 3: The *Unmerge stack* mechanism for token sequence length restoration. By applying *Unmerge* operations, the original length ($N = 784$) is restored to satisfy the spatial grid requirements of the convolutional pooling layer.

This mechanism allows the attention and Multilayer Perceptron (MLP) blocks to reap the computational benefits of processing a compressed token sequence, while ensuring that the convolutional pooling layers receive grid-aligned tokens. The mechanism introduces operational overhead associated with tracking the *Unmerge* functions and restoring the token sequence at the end of each stage.

3.2 Merging constraints

With the geometric requirements of the inter-stage convolutional pooling layers resolved by the *Unmerge stack*, we must now address the internal constraints of the Transformer blocks. Due to the SGA mechanism of SReT and the BSM algorithm of ToMe, the number of tokens merged at every iteration (r) cannot be set arbitrarily. As depicted in Figure 4, the token merging operation is inserted between the SGA and the MLP layers. Consequently, the reduction rate r must satisfy both the group divisibility constraints of the subsequent SGA operation and the BSM’s bipartite matching limits.

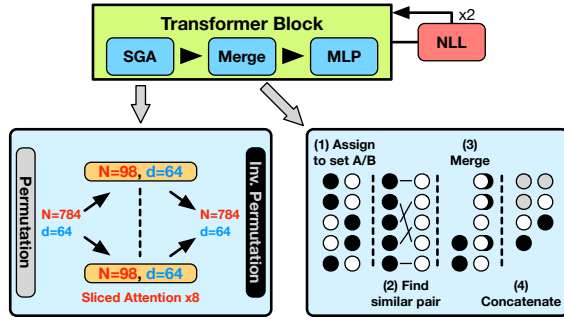


Figure 4: The MERGEOver Transformer block (SReT+ToMe), with the token merging module inserted between the Sliced Group Attention (SGA) and the Multilayer Perceptron (MLP) layers. Detailed expansions illustrate the spatial permutations and grouped attention within SGA, alongside the bipartite matching steps used to merge tokens. The Non-Linear projection Layer (NLL) is part of the recursion.

Within the SGA, the token sequence is divided equally into groups. As illustrated in Figure 4, attention is computed separately for each group. Thus, a strict constraint is that the sequence length entering the block must be evenly divisible by the designated *group number* (g) for that iteration, and there must be at least one token per group. For example, an incoming sequence length of $N = 784$ complies perfectly with a group number of $g = 8$, as each group receives exactly 98 tokens (Figure 4). Conversely, sequence lengths of $N = 15$ or $N = 7$ would trigger immediate tensor shape mismatches.

Since SReT blocks are executed recursively, the compressed sequence length exiting the current iteration ($N - r$) is the input sequence for the next iteration. Furthermore, SReT defines a different, generally lower group number (g_2) for such next iterations to allow each group to access more tokens, expanding the receptive field of the attention mechanism. To ensure the compressed sequence satisfies the requirements of SGA across all recursive passes, $N - r$ must be divisible by the Least Common Multiple (LCM) of the group numbers ($g_1, g_2, \dots$). Moreover, ToMe executes bipartite matching by first dividing the tokens into two sets (Figure 4). Since a token from one set is merged into a target token from the other, the algorithm cannot merge more than half of the total available tokens in a single iteration. Therefore, r must also be upper bounded by $\lfloor N/2 \rfloor$, resulting in the merging constraints,

which can be formalised as:

$$(N - r) \bmod g = 0, \quad N - r \geq g, \quad g = \text{LCM}(g_1, g_2), \quad r \leq \lfloor N/2 \rfloor. \quad (3)$$

3.3 Spatial tracking

As tokens are merged across the recursive stages of SReT, a single token comes to represent an increasingly large number of original patches. While standard ViT and SReT architectures treat all tokens equally, ToMe introduces a parallel token-size vector (s) to track the accumulated *mass* of every token to ensure proportional feature aggregation. Proportional attention (Equation (2)) is achieved by scaling the Key matrix using s , prior to the Softmax operation. To integrate this into SReT, we initialise a *mass* tensor of ones at the beginning of each forward pass. This tensor is updated across all layers and merge operations, to track the exact weight of each token.

During the forward pass, redundant tokens are identified via BSM. Crucially, we evaluate token similarity using the unweighted Key matrix (K) extracted from the attention layer. Isolating the similarity metric from the token mass ensures that merges are determined purely by semantic feature similarity, preventing “heavy” tokens with high accumulated mass from dominating the matching process.

Once BSM determines the optimal pairs, we merge the tokens using a weighted average. Since the tokens accumulate varying “masses” (the number of original spatial patches they represent), applying a naive, unweighted average would create a representational imbalance. Specifically, it would allow single-patch tokens equal influence over the merged result as multi-patch tokens. To prevent this, we pre-weight the input feature tensor by its corresponding mass via element-wise multiplication. The merging function then applies a sum reduction to independently aggregate both the weighted features and the parallel mass metrics. Finally, dividing the aggregated feature tensor by the updated mass tensor yields the correct weighted average representation.

A final structural challenge arises from SReT’s use of token permutations. As depicted in Figure 4, SGA applies permutations and inverse permutations to the input sequence to shift the attention window across recursive passes, ensuring global token interaction. If the mass tensor is not permuted identically, the weights will misalign with their corresponding tokens, corrupting the proportional attention. By decoupling the token-mass tensor (s) and routing it in parallel through the exact same permutation and inverse permutation functions as the feature sequence, we guarantee that the mass metrics remain synchronised with their corresponding tokens.

3.4 Token reduction scheduling

The original ToMe framework utilises two token reduction schedules: *constant* and *linearly decreasing* [10]. These schedules are defined using an absolute token count over a global network-depth index. The constant schedule merges a fixed number of tokens (r) at every merge-enabled block:

$$r_d^{\text{const}} = r, \quad (4)$$

where r_d^{const} is the number of tokens removed at block depth d . The linearly decreasing schedule starts by merging $2r$ tokens in the first

layer and reduces this amount to 0 by the final layer:

$$r_d^{\text{lin}} = \left\lfloor 2r \left(1 - \frac{d-1}{D-1} \right) \right\rfloor, \quad d = 1, \dots, D, \quad (5)$$

where D is the total number of blocks over which token merging is applied. Thus, $r_1^{\text{lin}} = 2r$ and $r_D^{\text{lin}} = 0$. While both schedules remove nearly the same total number of tokens for a given value of r , they distribute it differently over network depth. We retain these schedules as canonical ToMe baselines.

To account for the changing sequence lengths of the SRtT pyramid, we additionally formulate token reduction schedules relative to the initial sequence length of each stage: *stage-wise exponential* and *stage-wise single-shot*. Let s denote the stage, N_s its initial sequence length, D_s its number of blocks, and $d = 1, \dots, D_s$ the local block-depth index.

The stage-wise exponential schedule requests progressively smaller reductions:

$$r_{s,d}^{\text{exp}} = \lfloor \rho_{\text{exp}} \cdot N_s \cdot \alpha^{d-1} \rfloor, \quad 0 < \alpha < 1, \quad (6)$$

where ρ_{exp} is the stage-initial reduction fraction and α is the decay factor. Larger values of α produce slower decay, whereas smaller values concentrate the requested reduction near the beginning of each stage.

Conversely, a stage-wise single-shot schedule performs token reduction only at the first block of each stage:

$$r_{s,d}^{\text{shot}} = \begin{cases} \lfloor \rho_{\text{shot}} \cdot N_s \rfloor, & d = 1, \\ 0, & d > 1, \end{cases} \quad (7)$$

where ρ_{shot} is the stage-initial reduction fraction. This schedule maintains a fixed sequence length throughout the remaining blocks of the stage. Both schedules reset at stage boundaries.

Since the canonical and stage-wise schedules differ in both parameterisation and depth indexing, comparisons between them only evaluate the scheduling trajectories. Figure 5 illustrates these differences in trajectory for the four schedules. For the global schedules (constant and linear), the reduction percentages relative to the initial starting sequence length of each stage (N_s) increase after each stage even though the absolute number of tokens removed remains identical (constant) or decreases (linear). Conversely, the stage-wise schedules show near identical trajectories for all stages. For every schedule, the requested reduction r_{target} is adjusted to the nearest feasible reduction r_{safe} satisfying the group-divisibility and bipartite-matching constraints of Equation (3).

4 Experimental setup

We detail the experimental setup used to evaluate our proposed approach.

4.1 Dataset

All experiments are evaluated on the standard ImageNet-1K validation dataset (ILSVRC 2012) [17], which contains 50 000 images distributed evenly across 1 000 object classes. We preprocess all images using the standard ViT pipeline: resizing images to a 256-pixel shorter edge, extracting a 224×224 centre crop, and normalising the pixel values using the standard channel-wise ImageNet mean and standard deviation.

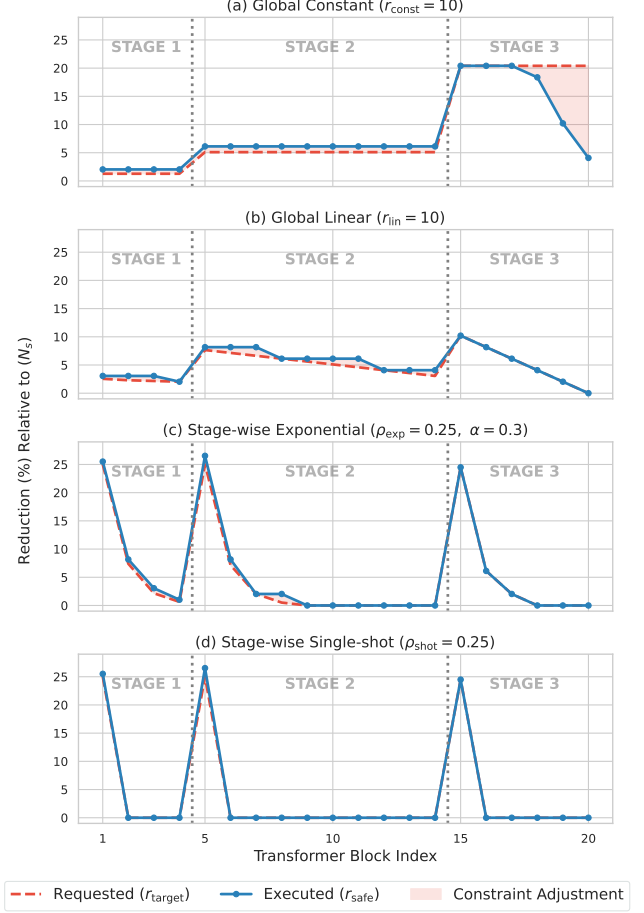


Figure 5: Normalised token reduction trajectories per schedule. Relative to the stage-initial sequence length (N_s), reductions increase at later stages for global schedules, while stage-wise schedules show near-identical trajectories throughout all stages. The constraint adjustments from r_{target} to r_{safe} illustrate the merging constraints of Equation 3.

4.2 Hardware and software environment

Evaluations are conducted using an NVIDIA GeForce RTX 4060 Ti GPU, an Intel Core Ultra 9 285K CPU (x86) with 64 GB of DDR-5 RAM, and a separate Raspberry Pi 5 (ARM) with 16 GB of RAM to demonstrate embedded deployment. The software stack consists of Python 3.10, CUDA 13.0, PyTorch 2.11.0, Torchvision 0.26.0, and timm 0.4.12. Theoretical complexity, measured in Floating-Point Operations (FLOPs), is computed using thop 0.1.1. Hardware throughput is measured using native CUDA timing events for GPU evaluations and custom performance snapshot utilities for CPU evaluations.

Specific execution parameters are enforced for accurate and comparative benchmarking. For GPU evaluations, cuDNN auto-tuning is enabled to optimise backend execution kernels. For CPU evaluations, strict GPU isolation is enforced, and computation is explicitly

constrained to four hardware threads. Moreover, MKLDNN is enabled for CPU environments.

4.3 Metrics

To evaluate the trade-offs introduced by the proposed integration, we measure hardware efficiency (latency (ms), throughput (img/s), and PAM (MB)), theoretical complexity (FLOPs (G) and parameter counts (M)), as well as task performance (Top-1 accuracy (%)).

We evaluate GPU throughput and PAM for Batch Sizes (BS) ranging from 1 to 128 in powers of 2. We simulate single-stream edge inference ($BS = 1$) and assess peak parallelisation ($BS = 128$), while evaluating the variations in performance for the intermediate batch sizes. For experiments involving a CPU, latency, throughput, and memory are evaluated for batch sizes ranging from 1 to 16 in powers of 2, to reflect edge-computing relevance.

GPU PAM is measured using PyTorch’s native CUDA allocator. CPU memory is reported separately as the change in process Resident Set Size (Δ RSS) observed during inference, relative to the pre-inference process state. Unlike GPU PAM, Δ RSS is a process-level measurement that includes tensor allocations, allocator caching, backend workspaces, and other resident memory. It therefore provides an estimate of changes in CPU process-memory usage rather than an isolated measurement of activation memory. Thus, the GPU and CPU memory values are not directly comparable and are only provided for reference.

We measure per-batch inference latency after a warm-up phase (20 iterations for GPU, 5 iterations for CPU), followed by evaluation iterations (100 iterations for GPU, 50 iterations for CPU). The median latency across the evaluation iterations is reported. Throughput is derived from the median batch latency, and thus latency and throughput are not independent measurements.

4.4 Baselines

We compare our results against the uncompressed SReT-Tiny-Distill [7], serving as a recursive hierarchical ViT baseline. Pre-trained weights for SReT are acquired directly from the authors’ public repository [25].

4.5 Token reduction scheduling strategies

To evaluate the impact of different token reduction scheduling strategies, we benchmark all four schedule types: *constant*, *linear*, *exponential*, and *single-shot*. We intentionally select the following configurations to expose the performance boundaries of recursive architectures:

- **Constant Schedule** (r_{const}): We evaluate the canonical ToMe global constant schedule at $r_{\text{const}} \in \{10, 20\}$, representing *light* and *heavy* reductions.
- **Linear Schedule** (r_{lin}): We evaluate the canonical ToMe global linear schedule at $r_{\text{lin}} \in \{10, 20\}$. These matched configurations test whether aggressively removing tokens early benefits hierarchical, recursive architectures compared to their constant counterparts.
- **Exponential Schedule** ($\rho_{\text{exp}}, \alpha$): We evaluate a *moderate reduction* ($\rho_{\text{exp}} = 0.25, \alpha = 0.3$) and an *aggressive reduction* ($\rho_{\text{exp}} = 0.40, \alpha = 0.3$). We fix α to isolate the effect of the

stage-initial reduction fraction while maintaining a common exponential trajectory.

- **Single-shot Schedule** (ρ_{shot}): We evaluate $\rho_{\text{shot}} \in \{0.25, 0.40\}$, matching the initial reduction fractions of the exponential configurations. This enables a direct comparison between the continued exponential reductions and stage-initial reductions followed by fixed sequence lengths.

5 Benchmarking results and discussion

Independent of the chosen processing platform (GPU, x86 CPU, ARM CPU) or batch size, Table 1 details the evaluation results for baseline model performance metrics. Table 5, Table 6, and Table 7 in Section A detail the performance metrics for all platforms and batch sizes.

Table 1: Platform and batch size independent model performance results on ImageNet-1K. Our selected configuration is highlighted. Percentage changes are relative to the SReT baseline.

Configuration	Top-1 Acc. (%)	Params. (M)	FLOPs (G)
<i>SReT baseline</i>	77.39	4.76	1.91
$r_{\text{const}} = 10$	70.95 -6.44	"	1.32 (-30.8%)
$r_{\text{const}} = 20$	40.97 -36.42	"	1.06 (-44.5%)
$r_{\text{lin}} = 10$	74.74 -2.65	"	1.46 (-23.5%)
$r_{\text{lin}} = 20$	14.76 -62.63	"	1.07 (-43.9%)
$\rho_{\text{exp}} = 0.25, \alpha = 0.3$	74.58 -2.81	"	1.35 (-29.3%)
$\rho_{\text{exp}} = 0.40, \alpha = 0.3$	66.28 -11.11	"	1.04 (-45.4%)
$\rho_{\text{shot}} = 0.25$	75.92 -1.47	"	1.49 (-22.0%)
$\rho_{\text{shot}} = 0.40$	72.36 -5.03	"	1.25 (-34.3%)

5.1 Token reduction schedules on GPU

Table 2 and Figure 6 summarise the throughput and PAM results for all tested configurations for batch sizes 1 and 16. At batch size 16, the global reduction schedules (constant and linear) do not improve throughput and show minimal reductions in PAM, while the stage-wise reduction schedules (exponential and single-shot) show meaningful improvements. Since the global and stage-wise schedules differ in both parameterisation and depth indexing, we do not attribute these outcomes to schedule shape alone. As illustrated in Figure 5, the stage-wise schedules express reductions relative to each stage’s initial sequence length and reset at stage boundaries, whereas the global schedules operate using absolute token counts over global network depth. Nonetheless, it is observed that between the global schedules, the linear schedule slightly outperforms the constant schedule.

At batch size 1, however, MERGEOver degrades GPU throughput for every tested configuration. This behaviour is consistent with the GPU requiring sufficient parallel work to amortise the added overhead of MERGEOver. At higher batch sizes, this overhead can be distributed across a larger amount of parallel computation, while, during single-stream inference, this overhead is more prominent. This batch-dependent crossover is observed across the batch sizes

Table 2: GPU performance on ImageNet-1K at batch sizes 1 and 16. Throughput (ThP) and Peak Activation Memory (PAM) are measured on an NVIDIA GeForce RTX 4060 Ti. Our selected configuration is highlighted. Percentage changes are relative to the SReT baseline.

Configuration	Batch size 1		Batch size 16	
	ThP (img/s)	PAM (MB)	ThP (img/s)	PAM (MB)
<i>SReT baseline</i>	223.83	6.22	1304.58	99.47
$r_{\text{const}} = 10$	110.24 (-50.7%)	6.01 (-3.3%)	1163.09 (-10.8%)	96.50 (-3.0%)
$r_{\text{const}} = 20$	109.88 (-50.9%)	5.91 (-4.9%)	1177.86 (-9.7%)	94.95 (-4.5%)
$r_{\text{lin}} = 10$	111.56 (-50.2%)	5.91 (-4.9%)	1218.04 (-6.6%)	94.95 (-4.5%)
$r_{\text{lin}} = 20$	117.12 (-47.7%)	5.70 (-8.3%)	1227.17 (-5.9%)	91.89 (-7.6%)
$\rho_{\text{exp}} = 0.25, \alpha = 0.3$	135.80 (-39.3%)	3.90 (-37.3%)	1597.22 (+22.4%)	61.27 (-38.4%)
$\rho_{\text{exp}} = 0.40, \alpha = 0.3$	136.99 (-38.8%)	3.90 (-37.3%)	1770.14 (+35.7%)	48.94 (-50.8%)
$\rho_{\text{shot}} = 0.25$	175.33 (-21.7%)	3.90 (-37.3%)	1588.00 (+21.7%)	61.27 (-38.4%)
$\rho_{\text{shot}} = 0.40$	174.42 (-22.1%)	3.90 (-37.3%)	1985.57 (+52.2%)	48.94 (-50.8%)

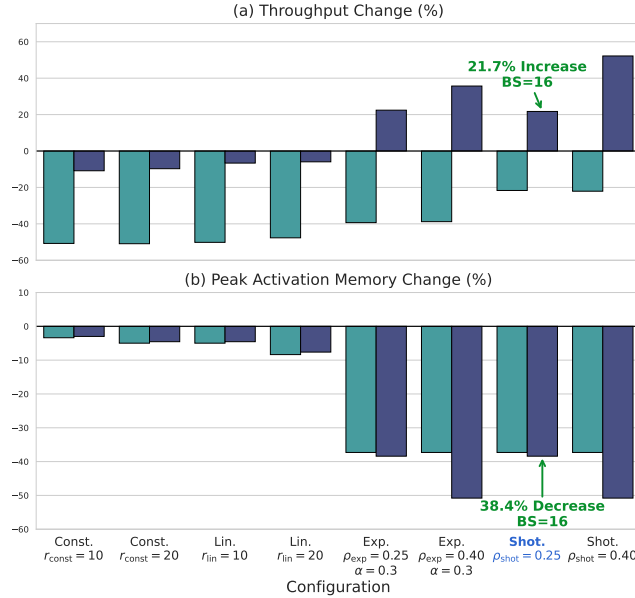


Figure 6: GPU performance results compared to the SReT baseline. Batch sizes 1 and 16 are represented with light and dark shades, respectively. Our chosen configuration shows a 21.7% increase in throughput and a 38.4% decrease in PAM for BS = 16.

from 1 to 128, with the detailed results reported in Table 5 in Section A. Low-level profiling and kernel optimisation are therefore warranted as future work.

Furthermore, Table 1 shows that a configuration with fewer FLOPs does not necessarily achieve higher throughput. The $\rho_{\text{exp}} = 0.40, \alpha = 0.3$ configuration has an estimated cost of 1.04 GFLOPs and a throughput increase of 35.7% at a batch size of 16. In contrast, the $\rho_{\text{shot}} = 0.40$ configuration shows a higher throughput increase of 52.2% despite having a higher estimated cost of 1.25 GFLOPs. The FLOPs estimate does not adequately represent costs such as BSM matching, token-mass tracking, and *Unmerge* operations. Moreover,

the single-shot schedule invokes token merging in fewer blocks than the exponential schedule. We therefore treat FLOPs as an approximate measure of tensor arithmetic rather than a predictor of hardware performance.

When evaluating the performance of the schedules, we adopt a maximum top-1 accuracy loss of 2 percentage points as an accuracy-preserving criterion. Under this criterion, we select the stage-wise single-shot configuration $\rho_{\text{shot}} = 0.25$, which reduces top-1 accuracy by 1.47 percentage points. However, this does not imply that the configuration is globally optimal. More aggressive configurations result in greater hardware performance at the cost of larger accuracy drops. The selected configuration reduces GPU throughput by 21.7% at batch size 1, while reducing PAM by 37.3%. At batch size 16, it increases throughput by 21.7% and reduces PAM by 38.4%. We also note that, in the evaluated configurations for the GPU, PAM is determined primarily by the initial reduction fraction (ρ_{exp} or ρ_{shot}), as the largest activation occurs near the beginning of each stage and subsequent reductions operate on already shortened sequences. This explains why exponential and single-shot schedules with the same initial reduction fraction produce similar PAM. MERGEOver therefore provides a favourable efficiency trade-off under batched execution, but does not accelerate single-stream GPU inference.

5.2 Token reduction schedules on x86 CPU

Table 3 and Figure 7 summarise the latency and process-memory results on the x86 CPU. In contrast to the GPU results, the stage-wise schedules improve latency at both batch sizes 1 and 16. At batch size 1, the constant and linear schedules with $r = 10$ increase latency by 5.4% and 7.6%, respectively, while their more aggressive variants provide reductions of only 0.1% and 3.1%. By comparison, the stage-wise schedules reduce latency by 13.6–23.0%. This indicates that the single-stream overhead observed on the GPU does not transfer directly to CPU execution and that the efficiency of MERGEOver is platform-dependent.

Table 3: x86 CPU performance on ImageNet-1K at batch sizes 1 and 16. Latency (Lat) and process-memory change (Δ RSS) are measured on an Intel Core Ultra 9 285K using four threads. Our selected configuration is highlighted. Percentage changes are relative to the SReT baseline.

Configuration	Batch size 1		Batch size 16	
	Lat (ms)	Δ RSS (MB)	Lat (ms)	Δ RSS (MB)
<i>SReT baseline</i>	14.94	19.94	144.32	92.97
$r_{\text{const}} = 10$	15.75 (+5.4%)	19.35 (-3.0%)	122.95 (-14.8%)	91.97 (-1.1%)
$r_{\text{const}} = 20$	14.92 (-0.1%)	19.77 (-0.9%)	109.90 (-23.8%)	90.08 (-3.1%)
$r_{\text{lin}} = 10$	16.08 (+7.6%)	22.60 (+13.3%)	124.76 (-13.6%)	90.41 (-2.8%)
$r_{\text{lin}} = 20$	14.47 (-3.1%)	22.07 (-10.7%)	104.43 (-27.6%)	79.02 (-15.0%)
$\rho_{\text{exp}} = 0.25, \alpha = 0.3$	12.91 (-13.6%)	21.28 (+6.7%)	102.65 (-28.9%)	109.37 (+17.6%)
$\rho_{\text{exp}} = 0.40, \alpha = 0.3$	12.18 (-18.5%)	21.13 (+6.0%)	71.34 (-50.6%)	99.77 (+7.3%)
$\rho_{\text{shot}} = 0.25$	12.27 (-17.9%)	22.32 (+11.9%)	100.98 (-30.0%)	72.52 (-22.0%)
$\rho_{\text{shot}} = 0.40$	11.51 (-23.0%)	21.13 (+6.0%)	77.43 (-46.3%)	99.52 (+7.0%)

At batch size 16, all evaluated schedules reduce latency. The global schedules achieve reductions of 13.6–27.6%, whereas the stage-wise schedules achieve reductions of 28.9–50.6%. However,

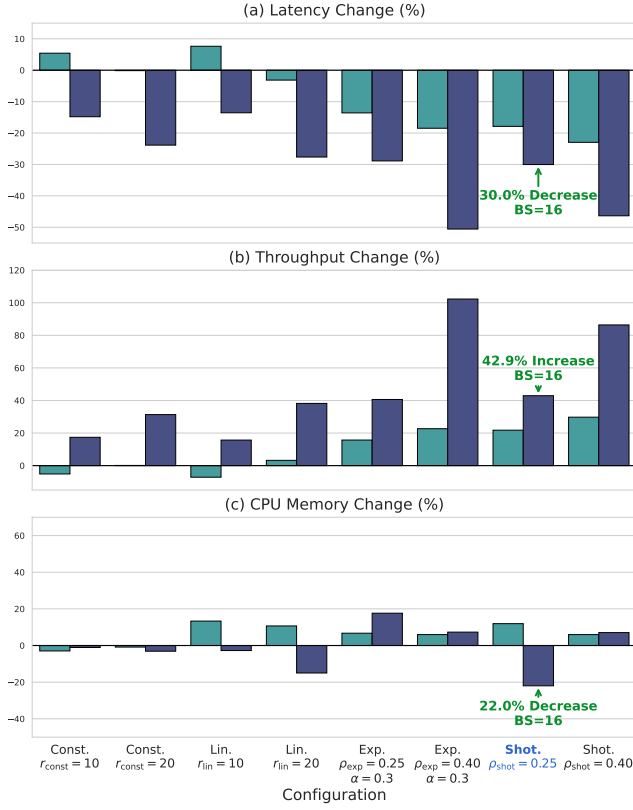


Figure 7: x86 CPU performance results compared to the SReT baseline. Batch sizes 1 and 16 are represented with light and dark shades, respectively. Our chosen configuration shows a 30.0% decrease in latency, a 42.9% increase in throughput, and a 22.0% decrease in process-memory change (Δ RSS) for $BS = 16$.

these hardware results must be considered alongside the accuracy results in Table 1. In particular, the larger latency reductions of $r_{\text{const}} = 20$, $r_{\text{lin}} = 20$, and the configurations using a reduction fraction of 0.40 come with substantial accuracy losses. Consequently, latency reduction alone does not determine the preferred configuration.

Under our accuracy-preserving criterion, the selected stage-wise single-shot configuration $\rho_{\text{shot}} = 0.25$ reduces latency by 17.9% at batch size 1 and by 30.0% at batch size 16. Unlike on the GPU, MERGEOver accelerates both single-stream and batched inference on the x86 CPU. The larger improvement at batch size 16 follows the pattern that the relative cost of token matching and sequence manipulation decreases as the amount of inference computation increases. Since we did not perform low-level profiling or optimisations, we treat this explanation as a hypothesis rather than a confirmed cause.

The reported Δ RSS values measure changes in process resident memory rather than peak activation memory. Their behaviour across schedules and batch sizes do not follow a strict pattern. For example, the selected configuration increases Δ RSS by 11.9% at

batch size 1 but reduces it by 22.0% at batch size 16. Similar inconsistencies occur among the other configurations. Due to the known limitations of RSS, we do not use our CPU memory measurements to rank the schedules or attribute these changes directly to token reduction.

5.3 Token reduction schedules on ARM CPU

Table 4 and Figure 8 summarise the latency and process-memory results on the Raspberry Pi 5. At batch size 1, the light global schedules increase latency by 11.8% for $r_{\text{const}} = 10$ and 11.6% for $r_{\text{lin}} = 10$. Their more aggressive counterparts remain close to the baseline, producing a 1.2% increase and a 1.3% reduction, respectively. The stage-wise schedules perform more favourably. The exponential and single-shot configurations with an initial reduction fraction of 0.25 reduce latency by 1.3% and 2.4%, respectively. More substantial reductions are obtained with a fraction of 0.40, but these configurations also come with larger accuracy losses.

Table 4: ARM CPU performance on ImageNet-1K at batch sizes 1 and 16. Latency (Lat) and process-memory change (Δ RSS) are measured on a Raspberry Pi 5 with 16 GB of RAM. Our selected configuration is highlighted. Percentage changes are relative to the SReT baseline.

Configuration	Batch size 1		Batch size 16	
	Lat (ms)	Δ RSS (MB)	Lat (ms)	Δ RSS (MB)
SReT baseline	142.82	21.89	1236.71	163.23
$r_{\text{const}} = 10$	159.61 (+11.8%)	24.86 (+13.6%)	1145.12 (-7.4%)	135.69 (-16.9%)
$r_{\text{const}} = 20$	144.50 (+1.2%)	24.16 (+10.4%)	1011.14 (-18.2%)	143.36 (-12.2%)
$r_{\text{lin}} = 10$	159.41 (+11.6%)	23.88 (+9.1%)	1167.84 (-5.6%)	142.53 (-12.7%)
$r_{\text{lin}} = 20$	140.90 (-1.3%)	23.97 (+9.5%)	965.58 (-21.9%)	136.25 (-16.5%)
$\rho_{\text{exp}} = 0.25$, $\alpha = 0.3$	140.94 (-1.3%)	22.83 (+4.3%)	975.44 (-21.1%)	85.98 (-47.3%)
$\rho_{\text{exp}} = 0.40$, $\alpha = 0.3$	126.84 (-11.2%)	22.73 (+3.8%)	774.81 (-37.3%)	61.89 (-62.1%)
$\rho_{\text{shot}} = 0.25$	139.42 (-2.4%)	22.75 (+3.9%)	1018.61 (-17.6%)	98.66 (-39.6%)
$\rho_{\text{shot}} = 0.40$	129.49 (-9.3%)	22.23 (+1.6%)	860.18 (-30.4%)	64.33 (-60.6%)

At batch size 16, all evaluated schedules reduce latency. The global schedules provide reductions between 5.6% and 21.9%, while the stage-wise schedules provide reductions between 17.6% and 37.3%. As with the x86 results, the most aggressive configuration does not necessarily provide the most favourable accuracy-latency trade-off. In particular, $\rho_{\text{exp}} = 0.40$, $\alpha = 0.3$ achieves the largest latency reduction of 37.3%, but reduces top-1 accuracy by 11.11 percentage points.

Under our accuracy-preserving criterion, the selected stage-wise single-shot configuration $\rho_{\text{shot}} = 0.25$ reduces latency by 2.4% at batch size 1 and by 17.6% at batch size 16. The limited improvement at batch size 1 shows that MERGEOver provides only a modest benefit for single-image inference on the Raspberry Pi. Its larger benefit at batch size 16 follows the cross-platform pattern that token reduction becomes more favourable under batched execution, where the computational savings can more effectively offset the additional merging and tracking operations.

Similar to the x86 results, the ARM Δ RSS measurements also depend strongly on batch size. Every evaluated configuration increases Δ RSS at batch size 1, whereas all configurations reduce it at batch size 16. For the selected configuration, Δ RSS increases by 3.9% at batch size 1 and decreases by 39.6% at batch size 16.

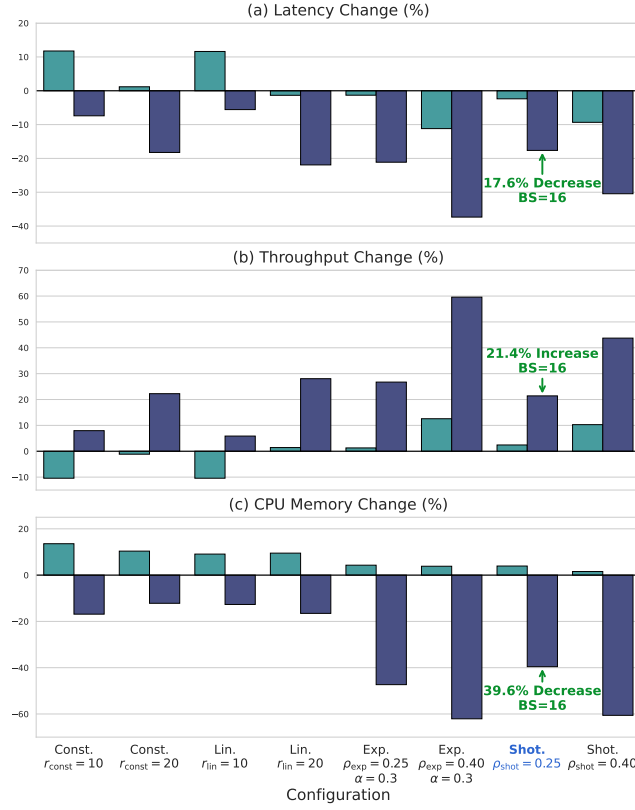


Figure 8: ARM CPU performance results compared to the SReT baseline. Batch sizes 1 and 16 are represented with light and dark shades, respectively. Our chosen configuration shows a 17.6% decrease in latency, a 21.4% increase in throughput, and a 39.6% decrease in process-memory change (ARSS) for $BS = 16$.

6 Limitations and future work

While we successfully demonstrate the viability of post-training dynamic token reduction in recursive architectures, several limitations present clear directions for future work.

First, the evaluation is restricted to SReT-Tiny-Distill. Consequently, we cannot separate effects caused by recursive weight sharing from those caused by SReT’s hierarchical pyramid or other architecture-specific properties. Evaluating MERGEOver on larger recursive models, non-recursive hierarchical backbones, and standard ViTs with ToMe is necessary to establish generalisability.

Second, the canonical and stage-wise schedules differ in parameterisation, depth indexing, and reduction trajectory. Since we do not evaluate matched per-stage fractional constant and linear schedules under equivalent reduction budgets, our comparisons evaluate the complete scheduling formulations and do not isolate schedule shape alone. Thus, they do not establish that the observed schedule behaviour is specific to recursive architectures.

GPU PAM is measured using the CUDA allocator, whereas CPU memory is measured as a change in process Resident Set Size (ΔRSS). The latter includes allocator caching, backend workspaces, and

physical page-residency behaviour and does not isolate activation memory. CPU memory results are therefore provided only as process-level observations and are not directly comparable to GPU PAM.

Furthermore, the thop FLOPs estimates do not fully capture BSM matching, token-mass tracking, sequence manipulation, or *Unmerge* tracking. We also do not perform low-level profiling or optimisation. Consequently, the causes of the observed platform- and batch-dependent performance differences cannot be isolated.

Finally, MERGEOver is evaluated relative to an uncompressed recursive baseline and does not establish superiority over conventional efficient ViTs. The modest Raspberry Pi latency improvement at batch size 1 demonstrates that practical single-stream edge deployment remains challenging. Future work could combine MERGEOver with low-level kernel optimisation and complementary post-training techniques, such as quantisation.

7 Conclusion

We presented MERGEOver and demonstrated the feasibility of combining the recursive weight-sharing of SReT with the dynamic token merging of ToMe as a post-training, multi-axis compression approach. By resolving the structural incompatibilities between hierarchical recursive architectures and dynamic sequence reduction, our approach enables token merging to be applied without retraining. Benchmarked on ImageNet-1K, our selected stage-wise single-shot configuration reduces top-1 accuracy by only 1.47 percentage points, which is negligible in many use-cases.

Considering the results from Section 5, the top performer scheduling strategy configuration, in terms of improved throughput, PAM and latency, while preserving the model accuracy, is the $\rho_{shot} = 0.25$. The achieved throughput and PAM improvements for batch size 16 are 21.7% increase and 38.4% decrease on the GPU platform. Focusing on the latency improvements for x86 and ARM CPU variants, we observe 30.0% reduction and 17.6% reduction, respectively, similarly for batch size 16.

These results demonstrate that the hardware benefits of token reduction depend strongly on the processing platform and batch size. While there can be setups with minimal improvements, certain batch sizes, e.g., 16, show immensely better batch processing performance. Though further algorithmic and experimental refinements are warranted, MERGEOver provides a post-training baseline for integrating token merging into hierarchical recursive Transformers and establishes a foundation for further low-level optimisation and evaluation, especially on resource-constrained hardware.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS’17)*. Curran Associates Inc., Long Beach, California, USA, 6000–6010. ISBN: 9781510860964. doi:10.5555/3295222.3295349.
- [2] Alexey Dosovitskiy et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*. OpenReview.net. <https://openreview.net/forum?id=YicbFdNTTy>.
- [3] Sachin Mehta and Mohammad Rastegari. 2022. MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer. (2022). arXiv: 2110.02178 [cs.CV]. doi:10.48550/arXiv.2110.02178.
- [4] Xinyu Liu, Houwen Peng, Ningxin Zheng, Yuqing Yang, Han Hu, and Yixuan Yuan. 2023. EfficientViT: Memory Efficient Vision Transformer with Cascaded

- Group Attention. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 14420–14430. doi:10.1109/CVPR52729.2023.01386.
- [5] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2019. Universal Transformers. (2019). arXiv: 1807.03819 [cs.CL]. doi:10.48550/arXiv.1807.03819.
- [6] Jinnian Zhang, Houwen Peng, Kan Wu, Mengchen Liu, Bin Xiao, Jianlong Fu, and Lu Yuan. 2022. MiniViT: Compressing Vision Transformers with Weight Multiplexing. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 12135–12144. doi:10.1109/CVPR52688.2022.01183.
- [7] Zhiqiang Shen, Zechun Liu, and Eric Xing. 2022. Sliced Recursive Transformer. In *Computer Vision – ECCV 2022*. Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner, (Eds.) Springer Nature Switzerland, Cham, 727–744. ISBN: 978-3-031-20053-3. doi:10.1007/978-3-031-20053-3_42.
- [8] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. 2021. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification. In *Advances in Neural Information Processing Systems*. Vol. 34. Curran Associates, Inc., 1393–1405. https://proceedings.neurips.cc/paper_files/paper/2021/file/747d3443e319a22747fbb873e8b2f9f2-Paper.pdf.
- [9] Youwei Liang, Chongjian Ge, Zhan Tong, Yibing Song, Jue Wang, and Pengtao Xie. 2022. Not All Patches are What You Need: Expediting Vision Transformers via Token Reorganizations. (2022). arXiv: 2202.07800 [cs.CV]. doi:10.48550/arXiv.2202.07800.
- [10] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. 2023. Token Merging: Your ViT But Faster. (2023). arXiv: 2210.09461 [cs.CV]. doi:10.48550/arXiv.2210.09461.
- [11] Hoai-Chau Tran et al. 2024. Accelerating Transformers with Spectrum-Preserving Token Merging. In *Advances in Neural Information Processing Systems*. A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, (Eds.) Vol. 37. Curran Associates, Inc., 30772–30810. doi:10.52202/079017-0969.
- [12] Minchul Kim, Shangqian Gao, Yen-Chang Hsu, Yilin Shen, and Hongxia Jin. 2024. Token Fusion: Bridging the Gap between Token Pruning and Token Merging. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 1372–1381. doi:10.1109/WACV57701.2024.00141.
- [13] Pavan Kumar Anasosalu Vasu, James Gabriel, Jeff Zhu, Oncel Tuzel, and Anurag Ranjan. 2023. FastViT: A Fast Hybrid Vision Transformer using Structural Reparameterization. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 5762–5772. doi:10.1109/ICCV51070.2023.00532.
- [14] Ao Wang, Hui Chen, Zijia Lin, Jungong Han, and Guiguang Ding. 2024. Rep ViT: Revisiting Mobile CNN From ViT Perspective. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 15909–15920. doi:10.1109/CVPR52733.2024.01506.
- [15] Jianjian Cao, Chong Yu, Peng Ye, and Tao Chen. 2026. MADTP++: Bridge the Gap Between Token and Weight Pruning for Accelerating ViTs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 48, 5, 5180–5194. doi:10.1109/TPAMI.2025.3650545.
- [16] [SW] Junseo Kim, MergeOver Code version v1.0.0, Aug. 2026. doi:10.5281/zenodo.21888951.
- [17] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–255. doi:10.1109/CVPR.2009.5206848.
- [18] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 10012–10022. doi:10.1109/ICCV48922.2021.00986.
- [19] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. 2021. Pyramid Vision Transformer: A Versatile Backbone for Dense Prediction without Convolutions. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 548–558. doi:10.1109/iccv48922.2021.00061.
- [20] Byeongho Heo, Sangdoo Yun, Dongyoon Han, Sanghyuk Chun, Junsuk Choe, and Seong Joon Oh. 2021. Rethinking Spatial Dimensions of Vision Transformers. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 11916–11925. doi:10.1109/iccv48922.2021.01172.
- [21] Lu Yu and Wei Xiang. 2023. X-pruner: explainable pruning for vision transformers. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 24355–24363. doi:10.1109/cvpr52729.2023.02333.
- [22] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*. S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, (Eds.) Vol. 35. Curran Associates, Inc., 16344–16359. doi:10.52202/068431-1189.
- [23] Yang Lin, Tianyu Zhang, Peiqin Sun, Zheng Li, and Shuchang Zhou. 2022. FQ-ViT: Post-Training Quantization for Fully Quantized Vision Transformer. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*. Lud De Raedt, (Ed.) International Joint Conferences on Artificial Intelligence Organization, (July 2022), 1173–1179. doi:10.24963/ijcai.2022/164.
- [24] Xinqing Li, Xin He, Xindong Zhang, Ming-Ming Cheng, Lei Zhang, and Yun Liu. 2026. Towards Joint Quantization and Token Pruning of Vision-Language Models. (2026). arXiv: 2604.17320 [cs.CV]. doi:10.48550/arXiv.2604.17320.
- [25] Zhiqiang Shen. 2022. Sliced Recursive Transformer (SRt). <https://github.com/szsq0214/SRt>. GitHub repository, accessed 2026-04-28. (2022).

A Extended results

This appendix reports the complete batch-dependent measurements underlying the hardware-efficiency analysis. For each platform, we compare the recursive ViT baseline with the evaluated token-reduction schedules across all supported batch sizes. The tables report Latency (Lat), Throughput (ThP), Peak Activation Memory (PAM), and process Resident Set Size (ΔRSS) where applicable. Percentage changes shown next to the measurements are computed relative to the recursive ViT baseline.

Table 5 presents the GPU measurements, while Tables 6 and 7 report the corresponding results for the x86 and ARM CPU platforms, respectively.

Table 5: GPU performance on ImageNet-1K across all tested batch sizes (1–128). We report Throughput (ThP) and Peak Activation Memory (PAM) measured on an NVIDIA GeForce RTX 4060 Ti. Our selected configuration is highlighted. Percentage changes are relative to the SReT baseline.

Metric	Batch size							
	1	2	4	8	16	32	64	128
Recursive ViT – SReT-Tiny-Distill [7]								
ThP (img/s)	223.83	438.91	849.45	1213.81	1304.58	1256.95	1174.00	1076.17
PAM (MB)	6.22	12.43	24.87	49.73	99.47	199.91	397.88	795.76
<i>Constant reduction: $r_{\text{const}} = 10$</i>								
ThP (img/s)	110.24 (-50.7%)	221.19 (-49.6%)	435.09 (-48.8%)	772.07 (-36.4%)	1163.09 (-10.8%)	1324.10 (+5.3%)	1275.88 (+8.7%)	1180.51 (+9.7%)
PAM (MB)	6.01 (-3.3%)	12.03 (-3.2%)	24.06 (-3.2%)	48.30 (-2.9%)	96.50 (-3.0%)	192.51 (-3.7%)	385.02 (-3.2%)	769.79 (-3.3%)
<i>Constant reduction: $r_{\text{const}} = 20$</i>								
ThP (img/s)	109.88 (-50.9%)	217.68 (-50.4%)	428.69 (-49.5%)	781.68 (-35.6%)	1177.86 (-9.7%)	1475.31 (+17.4%)	1439.29 (+22.6%)	1340.50 (+24.6%)
PAM (MB)	5.91 (-4.9%)	11.82 (-4.9%)	23.63 (-5.0%)	47.55 (-4.4%)	94.95 (-4.5%)	189.15 (-5.4%)	380.27 (-4.4%)	756.22 (-5.0%)
<i>Linear reduction: $r_{\text{lin}} = 10$</i>								
ThP (img/s)	111.56 (-50.2%)	222.00 (-49.4%)	441.16 (-48.1%)	821.16 (-32.3%)	1218.04 (-6.6%)	1326.12 (+5.5%)	1275.77 (+8.7%)	1181.37 (+9.8%)
PAM (MB)	5.91 (-4.9%)	11.82 (-4.9%)	23.63 (-5.0%)	47.55 (-4.4%)	94.95 (-4.5%)	189.15 (-5.4%)	380.27 (-4.4%)	756.22 (-5.0%)
<i>Linear reduction: $r_{\text{lin}} = 20$</i>								
ThP (img/s)	117.12 (-47.7%)	224.51 (-48.8%)	440.79 (-48.1%)	816.08 (-32.8%)	1227.17 (-5.9%)	1554.19 (+23.6%)	1540.13 (+31.2%)	1440.20 (+33.8%)
PAM (MB)	5.70 (-8.3%)	11.40 (-8.3%)	22.80 (-8.3%)	46.06 (-7.4%)	91.89 (-7.6%)	183.15 (-8.4%)	366.76 (-7.8%)	729.46 (-8.3%)
<i>Exponential reduction: $\rho_{\text{exp}} = 0.25, \alpha = 0.3$</i>								
ThP (img/s)	135.80 (-39.3%)	268.45 (-38.8%)	534.41 (-37.1%)	1007.02 (-17.0%)	1597.22 (+22.4%)	1665.33 (+32.5%)	1626.84 (+38.6%)	1477.07 (+37.3%)
PAM (MB)	3.90 (-37.3%)	7.72 (-37.9%)	15.37 (-38.2%)	30.67 (-38.3%)	61.27 (-38.4%)	123.13 (-38.4%)	245.46 (-38.3%)	489.38 (-38.5%)
<i>Exponential reduction: $\rho_{\text{exp}} = 0.40, \alpha = 0.3$</i>								
ThP (img/s)	136.99 (-38.8%)	271.92 (-38.0%)	543.60 (-36.0%)	1055.32 (-13.1%)	1770.14 (+35.7%)	2142.17 (+70.4%)	2146.38 (+82.8%)	2017.02 (+87.4%)
PAM (MB)	3.90 (-37.3%)	7.72 (-37.9%)	15.37 (-38.2%)	30.67 (-38.3%)	48.94 (-50.8%)	97.88 (-51.0%)	195.76 (-50.8%)	391.51 (-50.8%)
<i>Single-shot reduction: $\rho_{\text{shot}} = 0.25$</i>								
ThP (img/s)	175.33 (-21.7%)	344.65 (-21.5%)	686.66 (-19.2%)	1263.14 (+4.1%)	1588.00 (+21.7%)	1573.91 (+25.2%)	1491.64 (+27.1%)	1364.47 (+26.8%)
PAM (MB)	3.90 (-37.3%)	7.72 (-37.9%)	15.37 (-38.2%)	30.67 (-38.3%)	61.27 (-38.4%)	123.13 (-38.4%)	245.46 (-38.3%)	489.38 (-38.5%)
<i>Single-shot reduction: $\rho_{\text{shot}} = 0.40$</i>								
ThP (img/s)	174.42 (-22.1%)	345.86 (-21.2%)	690.04 (-18.8%)	1324.94 (+9.2%)	1985.57 (+52.2%)	1944.38 (+54.7%)	1883.19 (+60.4%)	1716.86 (+59.5%)
PAM (MB)	3.90 (-37.3%)	7.72 (-37.9%)	15.37 (-38.2%)	30.67 (-38.3%)	48.94 (-50.8%)	97.88 (-51.0%)	195.76 (-50.8%)	391.51 (-50.8%)

Table 6: x86 CPU performance on ImageNet-1K across all tested batch sizes (1–16). We report Latency (Lat), Throughput (ThP), and change in process Resident Set Size (Δ RSS) measured on an Intel Core Ultra 9 285K using four threads. Our selected configuration is highlighted. Percentage changes are relative to the SReT baseline.

Metric	Batch size				
	1	2	4	8	16
Recursive ViT – SReT-Tiny-Distill [7]					
Lat (ms)	14.94	22.76	36.26	68.28	144.32
ThP (img/s)	66.93	87.86	110.30	117.17	110.86
ΔRSS (MB)	19.94	25.14	38.70	57.67	92.97
<i>Constant reduction: $r_{\text{const}} = 10$</i>					
Lat (ms)	15.75 (+5.4%)	24.23 (+6.5%)	38.21 (+5.4%)	61.36 (-10.1%)	122.95 (-14.8%)
ThP (img/s)	63.50 (-5.1%)	82.55 (-6.0%)	104.67 (-5.1%)	130.38 (+11.3%)	130.14 (+17.4%)
ΔRSS (MB)	19.35 (-3.0%)	23.98 (-4.6%)	36.36 (-6.0%)	59.36 (+2.9%)	91.97 (-1.1%)
<i>Constant reduction: $r_{\text{const}} = 20$</i>					
Lat (ms)	14.92 (-0.1%)	22.12 (-2.8%)	34.72 (-4.2%)	53.26 (-22.0%)	109.90 (-23.8%)
ThP (img/s)	67.01 (+0.1%)	90.43 (+2.9%)	115.21 (+4.5%)	150.22 (+28.2%)	145.59 (+31.3%)
ΔRSS (MB)	19.77 (-0.9%)	21.65 (-13.9%)	36.05 (-6.8%)	58.09 (+0.7%)	90.08 (-3.1%)
<i>Linear reduction: $r_{\text{lin}} = 10$</i>					
Lat (ms)	16.08 (+7.6%)	24.88 (+9.3%)	37.74 (+4.1%)	61.37 (-10.1%)	124.76 (-13.6%)
ThP (img/s)	62.17 (-7.1%)	80.39 (-8.5%)	106.00 (-3.9%)	130.36 (+11.3%)	128.25 (+15.7%)
ΔRSS (MB)	22.60 (+13.3%)	24.18 (-3.8%)	36.27 (-6.3%)	58.72 (+1.8%)	90.41 (-2.8%)
<i>Linear reduction: $r_{\text{lin}} = 20$</i>					
Lat (ms)	14.47 (-3.1%)	20.35 (-10.6%)	31.69 (-12.6%)	54.07 (-20.8%)	104.43 (-27.6%)
ThP (img/s)	69.10 (+3.2%)	98.28 (+11.9%)	126.21 (+14.4%)	147.96 (+26.3%)	153.21 (+38.2%)
ΔRSS (MB)	22.07 (+10.7%)	24.04 (-4.4%)	33.59 (-13.2%)	56.53 (-2.0%)	79.02 (-15.0%)
<i>Exponential reduction: $\rho_{\text{exp}} = 0.25$, $\alpha = 0.3$</i>					
Lat (ms)	12.91 (-13.6%)	20.02 (-12.0%)	32.65 (-10.0%)	53.52 (-21.6%)	102.65 (-28.9%)
ThP (img/s)	77.44 (+15.7%)	99.88 (+13.7%)	122.52 (+11.1%)	149.46 (+27.6%)	155.88 (+40.6%)
ΔRSS (MB)	21.28 (+6.7%)	24.04 (-4.4%)	42.20 (+9.0%)	67.15 (+16.4%)	109.37 (+17.6%)
<i>Exponential reduction: $\rho_{\text{exp}} = 0.40$, $\alpha = 0.3$</i>					
Lat (ms)	12.18 (-18.5%)	17.54 (-22.9%)	25.50 (-29.7%)	39.00 (-42.9%)	71.34 (-50.6%)
ThP (img/s)	82.10 (+22.7%)	114.06 (+29.8%)	156.87 (+42.2%)	205.14 (+75.1%)	224.26 (+102.3%)
ΔRSS (MB)	21.13 (+6.0%)	20.70 (-17.7%)	40.61 (+4.9%)	62.31 (+8.0%)	99.77 (+7.3%)
<i>Single-shot reduction: $\rho_{\text{shot}} = 0.25$</i>					
Lat (ms)	12.27 (-17.9%)	21.20 (-6.9%)	30.75 (-15.2%)	54.13 (-20.7%)	100.98 (-30.0%)
ThP (img/s)	81.49 (+21.8%)	94.32 (+7.4%)	130.07 (+17.9%)	147.79 (+26.1%)	158.45 (+42.9%)
ΔRSS (MB)	22.32 (+11.9%)	25.69 (+2.2%)	32.87 (-15.1%)	48.45 (-16.0%)	72.52 (-22.0%)
<i>Single-shot reduction: $\rho_{\text{shot}} = 0.40$</i>					
Lat (ms)	11.51 (-23.0%)	17.54 (-22.9%)	25.01 (-31.0%)	41.58 (-39.1%)	77.43 (-46.3%)
ThP (img/s)	86.84 (+29.7%)	114.04 (+29.8%)	159.94 (+45.0%)	192.42 (+64.2%)	206.63 (+86.4%)
ΔRSS (MB)	21.13 (+6.0%)	22.73 (-9.6%)	40.60 (+4.9%)	62.28 (+8.0%)	99.52 (+7.0%)

Table 7: ARM CPU performance on ImageNet-1K across all tested batch sizes (1–16). We report Latency (Lat), Throughput (ThP), and change in process Resident Set Size (Δ RSS) measured on a Raspberry Pi 5 with 16 GB of RAM. Our selected configuration is highlighted. Percentage changes are relative to the SReT baseline.

Metric	Batch size				
	1	2	4	8	16
Recursive ViT – SReT-Tiny-Distill [7]					
Lat (ms)	142.82	204.02	346.97	637.68	1236.71
ThP (img/s)	7.00	9.80	11.53	12.55	12.94
ΔRSS (MB)	21.89	28.23	50.91	82.64	163.23
<i>Constant reduction: $r_{\text{const}} = 10$</i>					
Lat (ms)	159.61 (+11.8%)	214.50 (+5.1%)	351.50 (+1.3%)	620.03 (-2.8%)	1145.12 (-7.4%)
ThP (img/s)	6.27 (-10.4%)	9.32 (-4.9%)	11.38 (-1.3%)	12.90 (+2.8%)	13.97 (+8.0%)
ΔRSS (MB)	24.86 (+13.6%)	31.41 (+11.3%)	53.61 (+5.3%)	85.42 (+3.4%)	135.69 (-16.9%)
<i>Constant reduction: $r_{\text{const}} = 20$</i>					
Lat (ms)	144.50 (+1.2%)	194.64 (-4.6%)	311.57 (-10.2%)	552.06 (-13.4%)	1011.14 (-18.2%)
ThP (img/s)	6.92 (-1.1%)	10.28 (+4.9%)	12.84 (+11.4%)	14.49 (+15.5%)	15.82 (+22.3%)
ΔRSS (MB)	24.16 (+10.4%)	30.67 (+8.6%)	53.47 (+5.0%)	85.06 (+2.9%)	143.36 (-12.2%)
<i>Linear reduction: $r_{\text{lin}} = 10$</i>					
Lat (ms)	159.41 (+11.6%)	216.47 (+6.1%)	353.83 (+2.0%)	616.37 (-3.3%)	1167.84 (-5.6%)
ThP (img/s)	6.27 (-10.4%)	9.24 (-5.7%)	11.30 (-2.0%)	12.98 (+3.4%)	13.70 (+5.9%)
ΔRSS (MB)	23.88 (+9.1%)	30.50 (+8.0%)	53.05 (+4.2%)	84.86 (+2.7%)	142.53 (-12.7%)
<i>Linear reduction: $r_{\text{lin}} = 20$</i>					
Lat (ms)	140.90 (-1.3%)	184.37 (-9.6%)	293.57 (-15.4%)	512.92 (-19.6%)	965.58 (-21.9%)
ThP (img/s)	7.10 (+1.4%)	10.85 (+10.7%)	13.63 (+18.2%)	15.60 (+24.3%)	16.57 (+28.1%)
ΔRSS (MB)	23.97 (+9.5%)	30.53 (+8.1%)	52.98 (+4.1%)	89.22 (+8.0%)	136.25 (-16.5%)
<i>Exponential reduction: $\rho_{\text{exp}} = 0.25$, $\alpha = 0.3$</i>					
Lat (ms)	140.94 (-1.3%)	186.13 (-8.8%)	298.38 (-14.0%)	526.78 (-17.4%)	975.44 (-21.1%)
ThP (img/s)	7.09 (+1.3%)	10.75 (+9.7%)	13.41 (+16.3%)	15.19 (+21.0%)	16.40 (+26.7%)
ΔRSS (MB)	22.83 (+4.3%)	27.81 (-1.5%)	43.38 (-14.8%)	66.62 (-19.4%)	85.98 (-47.3%)
<i>Exponential reduction: $\rho_{\text{exp}} = 0.40$, $\alpha = 0.3$</i>					
Lat (ms)	126.84 (-11.2%)	165.14 (-19.1%)	256.52 (-26.1%)	421.88 (-33.8%)	774.81 (-37.3%)
ThP (img/s)	7.88 (+12.6%)	12.11 (+23.6%)	15.59 (+35.2%)	18.96 (+51.1%)	20.65 (+59.6%)
ΔRSS (MB)	22.73 (+3.8%)	27.70 (-1.9%)	43.17 (-15.2%)	66.45 (-19.6%)	61.89 (-62.1%)
<i>Single-shot reduction: $\rho_{\text{shot}} = 0.25$</i>					
Lat (ms)	139.42 (-2.4%)	187.30 (-8.2%)	295.35 (-14.9%)	539.22 (-15.4%)	1018.61 (-17.6%)
ThP (img/s)	7.17 (+2.4%)	10.68 (+9.0%)	13.54 (+17.4%)	14.84 (+18.2%)	15.71 (+21.4%)
ΔRSS (MB)	22.75 (+3.9%)	27.70 (-1.9%)	43.36 (-14.8%)	66.20 (-19.9%)	98.66 (-39.6%)
<i>Single-shot reduction: $\rho_{\text{shot}} = 0.40$</i>					
Lat (ms)	129.49 (-9.3%)	166.75 (-18.3%)	267.28 (-23.0%)	460.08 (-27.9%)	860.18 (-30.4%)
ThP (img/s)	7.72 (+10.3%)	11.99 (+22.3%)	14.97 (+29.8%)	17.39 (+38.6%)	18.60 (+43.7%)
ΔRSS (MB)	22.23 (+1.6%)	27.50 (-2.6%)	43.06 (-15.4%)	66.38 (-19.7%)	64.33 (-60.6%)